From one small network to a real digit classifier.

① Deeper MLP → more hidden layers, more neurons in the layer.

② multiclass classification → ① if K classes, then K neurons in the output layer.

② How to convert real nos. into probabilities.

③ Training stability.

③ Real dataset and classification on that: MNIST data

④ weight initialization.

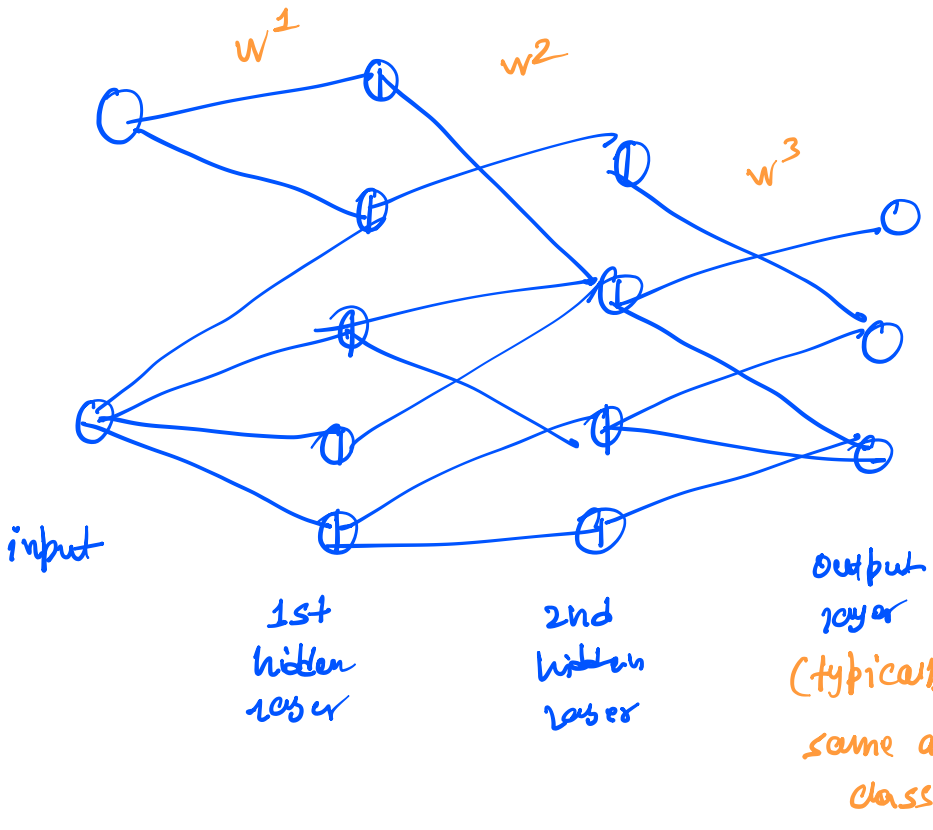
* **Deeper MLP:**

(2, 4, 1) ⇒ Three layer network having:
2 neurons in input layer
4 " " hidden "
1 neuron " output "

① (2, 8, 1)

② (2, 16, 8, 1)

(2, 16, 8, 1) is more capable. But it doesn't guarantee better validation score always. Depends on the data and pattern.



parameters

$W^1 = 2 \times 5 = 10$
$b^1 = 5$
} 15

$W^2 = 5 \times 4 = 20$
$b^2 = 4$
} 24

$W^3 = 4 \times 3 = 12$
$b^3 = 3$
} 15

Total parameters = $15 + 24 + 15 = 54$.

A layer with $n[l-1]$ inputs and $n[l]$ neurons, m is no. of samples

$$A^{[l-1]} \in \mathbb{R}^{m \times n_{l-1}}, \quad W^l \in \mathbb{R}^{n_{l-1} \times n_l}$$

$$b^l \in \mathbb{R}^{n_l}, \quad A^l \in \mathbb{R}^{m \times n_l}, \quad Z^l \in \mathbb{R}^{m \times n_l}$$

$\mathbb{R}$ → real nos.

$\mathbb{R}^n$ → vector of n numbers in Real no. space

$\mathbb{R}^{m \times n}$ → matrix $m \times n$ with nos. in Real space.

$$Z^l = A^{l-1} W^l + b^l$$

$$A^l = g^l(Z^l)$$

↳ activation function

count no. of parameters:

n^l → no. of neurons at layer l

L → Total no. of layers.

$$\text{Total parameters} = \sum_{l=1}^{L} n^{l-1} \times n^l + n^l$$

$2 \to 8 \to 1 \quad = \quad 33$ ✓

$2 \to 16 \to 8 \to 1 \quad = \quad 193$ ✓

* Multiclass Classification:

Rules of probs.

1. nos. should be b/w $(0, 1)$
2. The probabilities should always sum up to 1.
3. The relative ordering should be preserved.

Example:

$\mathbb{R}$: $\begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \to \begin{bmatrix} \quad \\ \quad \\ \quad \end{bmatrix}$: $2.0/3.1$, $1.0/3.1$, $0.1/3.1$ | $2.0/2.0 \to 1$, $1.0/2.0 \to 0.5$, $0.1/2.0 \to 0.05$

outputs from neurons → logits

fails of ≤ 0 values

Softmax: $\begin{bmatrix} e^{2.0} \\ e^{1.0} \\ e^{0.1} \end{bmatrix} \to \begin{bmatrix} \frac{e^{2.0}}{e^{2.0}+e^{1.0}+e^{0.1}} \\ \frac{e^{1.0}}{e^{2.0}+e^{1.0}+e^{0.1}} \\ \frac{e^{0.1}}{e^{2.0}+e^{1.0}+e^{0.1}} \end{bmatrix}$

$$P(y=k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

input, $y = f(x)$

$$\begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \xrightarrow{\text{Softmax}} \begin{bmatrix} 0.66 \\ 0.24 \\ 0.10 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$\hat{y}$ $\quad$ y

Multiclass cross-entropy.

Binary cross entropy

$$-y \cdot \log \hat{y} - (1-y) \cdot \log(1-\hat{y})$$

$-y \cdot \log \hat{y}$ → class 1 $\quad$ $(1-y) \cdot \log(1-\hat{y})$ → class 2

If true class is t.

$$L = -\log P(y=t \mid x)$$

with a one-hot vector,

$$L = -\sum_{k=1}^{K} y_k \log(\hat{p}_k)$$

True-class prob.	Loss	meaning
0.99	$-\log(0.99) \approx 0.01$	confident & correct
0.50	$-\log(0.5) \approx 0.69$	uncertainty
0.01	$-\log(0.01) \approx 4.61$	confidently wrong

Properties of loss function:

1. +ve number
2. increase when model makes higher Error.

10-class classification problem

1. model is untrained; uniform dist.

$$\begin{bmatrix} 0.1 \\ \vdots \\ 0.1 \end{bmatrix} \rightarrow -\ln(0.1) = 2.3$$

Caveats when we go deeper.

$$\frac{\partial l}{\partial w^1} = \frac{\partial l}{\partial A^L} \times \frac{\partial A^L}{\partial A^{L-1}} \times \frac{\partial A^{L-1}}{\partial A^{L-2}} \times \cdots \times \frac{\partial A^1}{\partial w^1}$$

$0.5^{10} = 0.00097$ $\quad$ $1.5^{10} = 57.66$

$\frac{\partial l}{\partial w_1}$ → vanishing gradient.

→ Exploding gradient

Dead-ReLU: -ve output receives zero gradient.
no weight update, neuron dead.